

Endpoint – Servizio Gestione Licenze v5

Documento aggiornato al 09/06/2026 — allineato all'analisi v4 + sezione 12 (decisione Alvisè).

Include: scopo, request body con campi commentati, controlli di validazione, risposte con JSON body esatti.

Convenzioni

Autenticazione client: `Authorization: Bearer <jwt> + X-License-Key: <license_key>`

Autenticazione vendor: `Authorization: Bearer <jwt_vendor>`

Formato errori:

```
{ "error_code": "SNAKE_CASE_COSTANTE", "message": "Descrizione leggibile", "details": { "field": "campo_opzionale" }, "timestamp": "ISO8601", "request_id": "uuid" }
```

SEZIONE CLIENT (C1–C7b)

C1 — POST /api/client/register

Scopo: Prima iscrizione del cliente al servizio. Verifica che la chiave prodotto esista e che il cliente non sia già registrato. Se è la prima registrazione, salva i dati in stato `pending` e invia un codice OTP via email per la verifica dell'indirizzo. Se il cliente è già registrato con licenza attiva, restituisce direttamente i dati della licenza (caso riapertura app — nessun OTP necessario).

Header: nessuno (endpoint pubblico — rate limited per IP)

Request body:

```
{ "product key": "string", // Chiave prodotto inclusa nella libreria client – obbligatorio "vat number": "string", // P.IVA o VAT number (formato libero per clienti esteri) – obbligatorio "company_name": "string", // Ragione sociale – obbligatorio "country": "string", // codice paese ISO 3166-1 alpha-2 (es. IT, DE, FR) – obbligatorio "contact_email": "string", // Email per notifiche, deve essere valida (RFC 5322) – obbligatorio "contact_phone": "string", // Telefono – opzionale "referent_name": "string" // Nome e cognome del referente – opzionale }
```

Tabelle DB coinvolte: `products` (lettura), `clients` (lettura/inserimento), `otp_codes` (inserimento), `rate_limits` (lettura/aggiornamento)

Controlli:

- `product_key` esiste in `products`
- `contact_email` in formato valido
- `vat_number` + `product_key` già presenti con licenza attiva → risposta 200 (nessun OTP)
- `vat_number` + `product_key` già presenti con trial scaduta e non rinnovata → risposta 409

- Rate limiting: max 5 richieste/ora per IP sorgente

Risposte:

```
// 201 - Prima registrazione, OTP inviato via email { "status": "pending_verification", "client_id": 42,
// id da passare a C2 e C3 "message": "Codice OTP inviato all'indirizzo email fornito" } // 206 - Gi
registrato con licenza attiva (riapertura app) { "status": "already_registered", "license_key":
"lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h", "license_type": "trial|monthly|annual", "expires at":
"2026-12-31T23:59:59Z", "modules": ["modulo_a", "modulo_b"] } // 400 - Campo obbligatorio mancante {
"error_code": "MISSING_REQUIRED_FIELD", "message": "Il campo product key e obbligatorio", "details": {
"field": "product_key" }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 406 - Email non
valida { "error_code": "INVALID_EMAIL_FORMAT", "message": "Il formato dell'email non è valido",
"details": { "field": "contact_email" }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } //
404 - Chiave prodotto non trovata { "error_code": "INVALID_PRODUCT_KEY", "message": "Chiave prodotto non
trovata", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 409 - Trial già utilizzata per
questo prodotto { "error_code": "TRIAL_ALREADY_USED", "message": "La trial per questo prodotto è già
stata utilizzata. Contattare il fornitore per attivare una licenza.", "timestamp":
"2026-06-09T10:00:00Z", "request_id": "uuid" } // 429 - Troppe richieste { "error_code":
"RATE_LIMIT_EXCEEDED", "message": "Troppi tentativi. Riprovare tra 1 ora.", "details": {
"retry_after_seconds": 3600 }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

C2 — POST /api/client/verify-otp

Scopo: Verifica il codice OTP ricevuto via email. Se valido: attiva il cliente (**status** → **active**), crea la licenza trial, genera la **license_key** univoca (HMAC-SHA256 + salt random), il JWT RS256 (TTL 60s), il refresh token (TTL 1h) e l'**offline_token** crittografato (AES-256-GCM). Imposta **vendor_synced = false** — la notifica O1 all'ERP è delegata al job schedulato **NEW_REGISTRATION** (non avviene in real-time, vedi sezione 12).

Idempotente: se C2 viene chiamata una seconda volta con gli stessi dati, restituisce gli stessi token senza creare duplicati.

Header: nessuno (endpoint pubblico)

Request body:

```
"client_id": 42, // id cliente ricevuto dalla risposta di C1 - obbligatorio "otp_code": "482931" //
codice OTP a 6 cifre ricevuto via email - obbligatorio
```

Tabelle DB coinvolte: **clients** (aggiornamento), **otp_codes** (lettura/eliminazione), **otp_attempts** (lettura/aggiornamento), **licenses** (inserimento), **client_tokens** (inserimento)

Controlli:

- **client_id** esiste in **clients** con **status = pending** (se **status = active** → idempotente, restituisce dati esistenti)
- **otp_code** corrisponde all'hash SHA256 salvato in **otp_codes**
- OTP non scaduto (TTL configurabile per prodotto)
- Max 3 tentativi falliti consecutivi → lockout 30 minuti (tabella **otp_attempts**)

Azioni in caso di successo:

1. **clients.status** → **active**
2. Genera **license_key** con HMAC-SHA256 + salt random
3. Crea licenza trial in **licenses** (durata e moduli da **products**)

4. Genera `offline_token` con AES-256-GCM
5. Genera JWT RS256 (TTL 60s) + refresh token (TTL 1h)
6. Imposta `licenses.vendor_synced = false`

Risposte:

```
// 201 - OTP verificato, trial attivata { "status": "trial activated", "license_key":  
"lk_6f8d3cia2b9e4f5c8a1b2c3d4e5f6g7h", // Salvare nella libreria client "license_type": "trial",  
"expires_at": "2026-12-31T23:59:59Z", "modules": [ "modulo_a", "modulo_b" ], "jwt":  
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."; "jwt_expires_in seconds": 60, "refresh_token":  
"rt_c3d4e5f6g7h8i9j0k1l2m3n4", // Salvare per C6 "offline_token":  
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."; // Salvare localmente "offline token expires at":  
"2026-06-18T15:30:00Z" } // 401 - OTP errato { "error_code": "INVALID_OTP", "message": "Codice OTP non  
valido", "details": { "attempts_remaining": 2 }, "timestamp": "2026-06-09T10:00:00Z", "request_id":  
"uuid" } // 401 - OTP scaduto { "error_code": "OTP_EXPIRED", "message": "Il codice OTP è scaduto.  
Richiedere un nuovo codice tramite C3.", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } //  
403 - Troppi tentativi falliti (lockout) { "error_code": "OTP_MAX_ATTEMPTS", "message": "Troppi  
tentativi falliti. Accesso bloccato per 30 minuti.", "details": { "retry_after_seconds": 1800 },  
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 404 - client_id non trovato {  
"error_code": "CLIENT_NOT_FOUND", "message": "ID cliente non trovato", "timestamp":  
"2026-06-09T10:00:00Z", "request_id": "uuid" }
```

C3 — POST /api/client/resend-otp

Scopo: Genera e invia un nuovo codice OTP quando quello precedente è scaduto. Invalida l'OTP precedente e ne crea uno nuovo con un nuovo TTL.

Header: nessuno (endpoint pubblico — rate limited per client)

Request body:

```
"client_id": 42 // ID cliente in stato pending — obbligatorio
```

Tabelle DB coinvolte: `clients` (lettura), `otp_codes` (aggiornamento), `rate_limits` (lettura/aggiornamento)

Controlli:

- `client_id` esiste con `status = pending`
- Rate limiting: max 3 richieste/ora per `client_id`

Risposte:

```
// 200 - Nuovo OTP inviato { "status": "otp_sent", "message": "Nuovo codice OTP inviato all'indirizzo  
email registrato" } // 404 - client_id non trovato o non in stato pending { "error_code":  
"CLIENT_NOT_FOUND", "message": "ID cliente non trovato o non in attesa di verifica", "timestamp":  
"2026-06-09T10:00:00Z", "request_id": "uuid" } // 429 - Troppe richieste { "error_code":  
"RATE_LIMIT_EXCEEDED", "message": "Troppi tentativi. Riprovare tra 1 ora.", "details": {  
"retry_after_seconds": 3600 }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

C4 — GET /api/client/license/status

Scopo: Check periodico dello stato della licenza. Restituisce stato corrente, moduli attivi e rinnova l'`offline_token`. La frequenza del check è configurabile per prodotto (`license_check_frequency_days`). Se la licenza risulta scaduta, C4 lo comunica al client ma non aggiorna il DB né chiama O1 — queste operazioni sono delegate al job schedulato `LICENSE_EXPIRED` (sezione 12).

Header:

```
Authorization: Bearer <jwt> X-License-Key: lk_6f8d3cia2b9e4f5c8a1b2c3d4e5f6g7h
```

Query parameters: nessuno

Tabelle DB coinvolte: `licenses` (lettura), `client_tokens` (verifica JWT), `products` (lettura frequenza), `client_activity_logs` (aggiornamento `last_c5_at`)

Controlli:

- JWT valido e non scaduto (RS256, TTL 60s) — se scaduto → 401, il client chiama C6
- `license_key` corrisponde al client nel payload JWT

Risposte:

```
// 200 - Licenza attiva { "status": "active", "license_type": "trial|monthly|annual", "expires_at":  
"2026-12-31T23:59:59Z", "modules": ["modulo a", "modulo b"], "offline token":  
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..", // Token aggiornato, da sovrascrivere  
"offline_token_expires_at": "2026-06-18T15:30:00Z", "next_check_in_days": 7 // D  
products.license_check_frequency_days } // 200 - Licenza scaduta (O1 e aggiornamento DB delegati al job  
LICENSE_EXPIRED) { "status": "expired", "license_type": "trial|monthly|annual", "expired_at":  
"2026-06-01T00:00:00Z" } // 200 - Licenza revocata { "status": "revoked", "revoked_at":  
"2026-06-05T12:00:00Z" } // 401 - JWT non valido o scaduto { "error_code": "INVALID_JWT", "message":  
"Token non valido o scaduto. Utilizzare C6 per rinnovarlo.", "timestamp": "2026-06-09T10:00:00Z",  
"request_id": "uuid" } // 404 - Licenza non trovata { "error_code": "LICENSE_NOT_FOUND", "message":  
"Licenza non trovata per la license key fornita", "timestamp": "2026-06-09T10:00:00Z", "request_id":  
"uuid" }
```

C5 — GET /api/client/messages

Scopo: Poll periodico per recuperare i messaggi in-app in coda (avvisi di scadenza, notifiche di rinnovo, comunicazioni del fornitore). Aggiorna `last_seen_at` nel log attività e segna i messaggi come consegnati (`delivered_at`).

Header:

```
Authorization: Bearer <jwt> X-License-Key: lk_..
```

Query parameters: nessuno

Tabelle DB coinvolte: `messages` (lettura/aggiornamento), `client_activity_logs` (aggiornamento)

Controlli:

- JWT valido

Risposte:

```
// 200 - Lista messaggi (array vuoto se non ci sono messaggi in coda) { "messages": [ { "id": 101,
"template_key": "SCADENZA_IMMINENTE", "content": "La tua licenza scade tra 7 giorni. Contatta il
fornitore per il rinnovo.", "created_at": "2026-06-02T10:00:00Z" } ] } // 401 - JWT non valido {
"error_code": "INVALID_JWT", "message": "Token non valido o scaduto. Utilizzare C6 per rinnovarlo.",
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

Scopo: Rinnova il JWT del client usando il refresh token. Il refresh token viene **ruotato** ad ogni utilizzo: il vecchio viene invalidato e ne viene emesso uno nuovo. Se il refresh token è scaduto (dopo 1h di inattività), il client deve contattare il produttore per ottenere un nuovo accesso.

Header: nessuno

Request body:

```
{ "refresh_token": "rt_c3d4e5f6g/h8i9j0k1l2" // Refresh token ricevuto da C2 o C6 precedente - obbligatorio
```

Tabelle DB coinvolte: `client_tokens` (lettura, aggiornamento rotation)

Controlli:

- `refresh_token` esiste in `client_tokens` ed è ancora valido (TTL 1h)
- Non già usato in precedenza (ogni refresh token è monouso)

Risposte:

```
// 209 - Nuovi token emessi { "jwt": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJ1b250dW8ucmVudCkiLCJpYXNjaWRfbm90b250dW8ucmVudCk6bnvuoquo",  
    "jwt_expires_in_seconds": 60, "refresh_token": "rt_nuovo_ruftato_abc123". // Il vecchio è invalidato -  
    salvare questo "refresh token expires in seconds": 3600 } // 401 - Refresh token non valido o già usato  
    { "error_code": "INVALID_REFRESH_TOKEN", "message": "Refresh token non valido o già utilizzato"  
    "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 401 - Refresh token scaduto (in  
    inattività) { "error_code": "REFRESH_TOKEN_EXPIRED", "message": "Sessione scaduta per inattività. I  
    cliente deve contattare il produttore.", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid"
```

Scopo: Avvia il cambio dell'indirizzo email del cliente. Invia un codice OTP alla nuova email per la verifica. Il cambio viene completato solo dopo la verifica tramite C7b.

Header:

```
Authorization: Bearer <jwt> X-License-Key: lk_...
```

Request body:

```
{ "new_email": "nuova@email.com" // Nuovo indirizzo email - obbligatorio, deve essere valido }
```

Tabelle DB coinvolte: `clients` (lettura), `otp_codes` (inserimento)

Controlli:

- JWT valido
- `new_email` in formato valido
- `new_email` diverso dall'email corrente del cliente
- `new_email` non già in uso da un altro cliente per lo stesso prodotto

Risposte:

```
// 200 - OTP inviato alla nuova email { "status": "otp_sent", "message": "Codice OTP inviato al nuovo indirizzo email. Verificare con C7b." } // 400 - Email non valida { "error_code": "INVALID_EMAIL_FORMAT", "message": "Il formato dell'email non è valido", "details": { "field": "new_email" }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 400 - Email uguale a quella corrente { "error_code": "EMAIL_SAME_AS_CURRENT", "message": "Il nuovo indirizzo email è uguale a quello attuale", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 401 - JWT non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 409 - Email già in uso { "error_code": "EMAIL_ALREADY_IN_USE", "message": "L'indirizzo email è già associato a un altro cliente", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

C7b — POST /api/client/verify-email-change (nuovo in v4)

Scopo: Verifica il codice OTP inviato alla nuova email (da C7) e completa il cambio email aggiornando il campo `contact_email` nel DB.

Header:

```
Authorization: Bearer <jwt> X-License-Key: lk_...
```

Request body:

```
"otp_code": "391847" // codice OTP a 6 cifre ricevuto sulla nuova email — obbligatorio
```

Tabelle DB coinvolte: `clients` (aggiornamento), `otp_codes` (lettura/eliminazione), `otp_attempts` (lettura/aggiornamento)

Controlli:

- JWT valido
- OTP valido, non scaduto, corrisponde all'hash SHA256 in `otp_codes`
- Max 3 tentativi falliti

Risposte:

```
// 200 - Email aggiornata con successo { "status": "email_changed", "new_email": "nuova@email.com" } // 401 - OTP errato { "error_code": "INVALID_OTP", "message": "Codice OTP non valido", "details": { "attempts_remaining": 1 }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 401 - OTP scaduto { "error_code": "OTP_EXPIRED", "message": "Il codice OTP è scaduto. Richiedere un nuovo cambio email tramite C7.", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 403 - Troppi tentativi falliti { "error_code": "OTP_MAX_ATTEMPTS", "message": "Troppi tentativi falliti. Riprovare tra 30 minuti.", "details": { "retry_after_seconds": 1800 }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

SEZIONE FORNITORE (F1–F9)

F1 — POST /api/vendor/auth/login

Scopo: Autenticazione del fornitore tramite API key statica. L'API key viene verificata tramite bcrypt (rounds=12) sull'hash salvato in DB. Emette JWT RS256 (TTL 60s) e refresh token (TTL 1h) per le chiamate successive. Rate limited su IP.

Header: nessuno

Request body:

```
"api_key": "vk_chiave_statica_del_fornitore" // API key assegnata al fornitore – obbligatorio
```

Tabelle DB coinvolte: `vendors` (lettura hash), `vendor_tokens` (inserimento), `rate_limits` (lettura/aggiornamento)

Controlli:

- `api_key` corrisponde all'hash bcrypt in `vendors.api_key_hash`
- API key non revocata (`api_key_revoked_at` IS NULL)
- Rate limiting: max 10 richieste/ora per IP sorgente

Risposte:

```
// 200 – Autenticazione OK { "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...", "jwt_expires_in_seconds":  
60, "refresh_token": "rt_vendor_abc123", "refresh_token_expires_in_seconds": 3600, "vendor_id": 1 } //  
401 – API key non valida { "error_code": "INVALID_API_KEY", "message": "API key non valida",  
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 401 – API key revocata (dopo F9) {  
"error_code": "API_KEY_REVOKED", "message": "API key revocata. Utilizzare la nuova chiave generata con  
F9.", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 429 – Troppe richieste {  
"error_code": "RATE_LIMIT_EXCEEDED", "message": "Troppi tentativi. Riprovare tra 1 ora.", "details":  
"retry_after_seconds": 3600 }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F2 — POST /api/vendor/token/refresh

Scopo: Rinnova il JWT del fornitore tramite refresh token (rotation). Stessa logica di C6 applicata al vendor.

Header: nessuno

Request body:

```
"refresh_token": "rt_vendor_abc123" // Refresh token ricevuto da F1 o F2 precedente – obbligatorio
```

Tabelle DB coinvolte: `vendor_tokens` (lettura, aggiornamento rotation)

Controlli:

- `refresh_token` valido e non scaduto (TTL 1h)

- Non già usato (monouso per rotation)

Risposte:

```
// 200 - Nuovi token emessi { "jwt": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."},
"jwt_expires_in_seconds": 60, "refresh_token": "rt_vendor_nuovo_ruotato", // il vecchio è invalidato
"refresh_token_expires_in_seconds": 3600 } // 401 - Refresh token non valido o scaduto { "error_code":
"INVALID_REFRESH_TOKEN", "message": "Refresh token non valido o scaduto. Eseguire nuovamente F1.",
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F3 — GET /api/vendor/registrations/new

Scopo: Restituisce la lista paginata delle nuove registrazioni clienti non ancora sincronizzate con l'ERP (`vendor_synced = false`). Il fornitore scarica i dati, li processa nel proprio sistema e poi chiama F4 per confermare la ricezione.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Query parameters:

```
page=1 // Numero di pagina — default 1 &limit=50 // Risultati per pagina — default 50, massimo 100
```

Tabelle DB coinvolte: `clients` (lettura), `licenses` (lettura, filtro `vendor_synced = false`), `products` (lettura)

Controlli:

- JWT vendor valido
- `page` e `limit` sono interi positivi; `limit` ≤ 100

Risposte:

```
// 200 - Lista registrazioni non sincronizzate { "data": [ { "registration_id": 42, // ID da passare
F4 per la conferma "vat_number": "IT12345678901", "company_name": "Acme Srl", "country": "IT",
"contact_email": "admin@acme.it", "contact_phone": "+39 02 1234567", "referent_name": "Mario Rossi",
"product_key": "PK-XYZ", "license_type": "trial", "registered_at": "2026-06-09T10:00:00Z" } ],
"pagination": { "page": 1, "limit": 50, "total": 1, "total_pages": 1 } } // 400 - Parametri di
paginazione non validi { "error_code": "INVALID_PAGE_PARAMETER", "message": "I parametri page e limit
devono essere interi positivi (limit max 100)", "timestamp": "2026-06-09T10:00:00Z", "request_id":
"uuid" } // 401 - JWT non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto.
Eseguire F1 o F2.", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F4 — POST /api/vendor/registrations/confirm

Scopo: Conferma che l'ERP ha ricevuto e processato le registrazioni scaricate con F3. Imposta `vendor_synced = true` per gli ID confermati, che non verranno più restituiti da F3. **Idempotente:** chiamare F4 più volte con gli stessi ID non produce errori né duplicati.

Header:


```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{ "registration_ids": [42, 43, 44] // Lista degli ID registrazione da marcare come sincronizzati  
  obbligatorio }
```

Tabelle DB coinvolte: `licenses` (aggiornamento `vendor_synced`)

Controlli:

- JWT vendor valido
- Tutti gli ID in `registration_ids` esistono e appartengono a questo vendor
- Se già confermati → no-op su quei record, risposta 200 comunque

Risposte:

```
// 200 - Conferma registrata { "status": "confirmed", "confirmed_ids": [42, 43], "already_synced_ids":  
  [44] // IDs già sincronizzati in precedenza (idempotenza) } // 401 - JWT non valido { "error_code":  
  "INVALID_JWT", "message": "Token non valido o scaduto", "timestamp": "2026-06-09T10:00:00Z",  
  "request_id": "uuid" } // 404 - Uno o più ID non trovati { "error_code": "REGISTRATION NOT FOUND",  
  "message": "Uno o più ID registrazione non trovati o non appartengono a questo vendor", "details": {  
    "invalid_ids": [99] }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F5 — POST /api/vendor/license/activate

Scopo: Attiva una licenza a pagamento (mensile o annuale) per un cliente già registrato. Sostituisce il contratto trial o provvisorio con uno standard. Supporta anche la creazione di licenze provvisorie (`is_provisional = true`) in attesa di conferma pagamento. **Idempotente tramite header `Idempotency-Key`:** in caso di timeout o doppio invio, restituisce la risposta originale dalla cache (TTL 24h, tabella `idempotency_keys`).

Header:

```
Authorization: Bearer <jwt_vendor> Idempotency-Key: 550e8408-e29b-41d4-a716-446655440000 // UUID univoco  
per questa operazione - obbligatorio
```

Request body:

```
{ "vat_number": "IT12345678901", // P.IVA del cliente - obbligatorio "product_key": "PK-XYZ", // Chiave  
  prodotto - obbligatorio "license_type": "monthly/annual", // tipo di licenza - obbligatorio "starts_at":  
  "2026-06-09T00:00:00Z", // Data inizio licenza - obbligatorio "expires_at": "2026-07-09T23:59:59Z", //  
  Data scadenza - obbligatorio "max_users": 5, // Numero massimo utenti "modules": ["modulo a",  
  "modulo b"], // Moduli abilitati - obbligatorio "is_provisional": false // true = licenza provvisoria  
  upgradabile tramite F5 }
```

Tabelle DB coinvolte: `clients` (lettura), `licenses` (aggiornamento vecchia, inserimento nuova), `idempotency_keys` (lettura/inserimento)

Controlli:

- JWT vendor valido
- `vat_number` + `product_key` esistono nel DB con cliente attivo

- `license_type` è `monthly` o `annual`
- `expires_at` > `starts_at`
- Header `Idempotency-Key`: se già processato → risposta dalla cache con `_cached: true`

Risposte:

```
// 201 - Licenza attivata { "status": "activated", "license_key": "lk_6f8d3cia2b9e4f5c8a1b2c3d4e5f6g7h",
"license_type": "monthly", "starts_at": "2026-06-09T00:00:00Z", "expires_at": "2026-07-09T23:59:59Z",
"modules": [ "modulo a", "modulo b" ], "is_provisional": false } // 200 - Risposta dalla cache
{ "idempotency-key già elaborata" { "status": "activated", "license_key":
"lk_6f8d3cia2b9e4f5c8a1b2c3d4e5f6g7h", "license_type": "monthly", "starts_at": "2026-06-09T00:00:00Z",
"expires_at": "2026-07-09T23:59:59Z", "modules": [ "modulo a", "modulo b" ], "is_provisional": false,
"_cached": true } // 400 - Date non valide { "error_code": "INVALID_DATE_RANGE", "message": "expires_at
deve essere successivo a starts_at", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 400
- idempotency-key mancante { "error_code": "MISSING_IDEMPOTENCY_KEY", "message": "L'header
idempotency-key è obbligatorio per questo endpoint", "timestamp": "2026-06-09T10:00:00Z", "request_id":
"uuid" } // 401 - JWT non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto",
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 404 - Cliente non trovato { "error_code":
"CLIENT_NOT_FOUND", "message": "Nessun cliente trovato per vat_number e product_key forniti",
"timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F6 — POST /api/vendor/products

Scopo: Registra un nuovo prodotto nel sistema. Genera la `product_key` univoca da includere nella libreria client distribuita ai clienti del fornitore.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{ "product_name": "MioSoftware Pro", // Nome del prodotto — obbligatorio "trial_duration_days": 30, //
Durata trial in giorni — obbligatorio "trial_max_users": 1, // Max utenti durante la trial —
obbligatorio "trial_modules": [ "modulo a" ], // Moduli inclusi nella trial — obbligatorio
"license_check_frequency_days": 7 // Frequenza check C4 in giorni — obbligatorio }
```

Tabelle DB coinvolte: `products` (inserimento), `modules` (inserimento/associazione)

Controlli:

- JWT vendor valido
- `product_name` non vuoto
- `trial_duration_days` > 0
- `license_check_frequency_days` > 0

Risposte:

```
// 201 - Prodotto registrato { "product key": "PK-A1B2C3D4", // Chiave da includere nella libreria
Client "product_name": "MioSoftware Pro", "trial_duration_days": 30, "trial_max_users": 1,
"trial_modules": [ "modulo a" ], "license_check_frequency_days": 7 } // 400 - Campo obbligatorio mancante
{ "error_code": "MISSING_REQUIRED_FIELD", "message": "Il campo product_name è obbligatorio", "details":
{ "field": "product_name" }, "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 401 - JWT
non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto", "timestamp":
```

```
"2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F7 — POST /api/vendor/client/billing

Scopo: Salva i dati di fatturazione del cliente (raccolti dall'ERP al primo acquisto). Operazione upsert: se i dati esistono già vengono aggiornati. **Idempotente per natura** (upsert). L'IBAN è opzionale: necessario solo se il fornitore utilizza addebito diretto SEPA; altrimenti il pagamento è gestito interamente nell'ERP.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{ "vat_number": "IT12345678901", // P.IVA del cliente = obbligatorio "product_key": "PK-XYZ", // Prodotto di riferimento - obbligatorio "pec_email": "admin@acme.pec.it", // PEC per fatturazione elettronica - obbligatorio se IT "sdi_code": "ABC1234", // Codice destinatario SDI - alternativo a PEC se IT "billing_address": "Via Roma 1", // indirizzo di fatturazione - obbligatorio "billing_city": "Milano", // Città - obbligatorio "billing_zip": "20121", // CAP - obbligatorio "billing_country": "IT", // Paese ISO = obbligatorio "iban": "IT60X0542811101000000123456" // IBAN = opzionale, solo se pagamento SEPA }
```

Tabelle DB coinvolte: `clients` (lettura), `client_billing` (upsert)

Controlli:

- JWT vendor valido
- `vat_number` + `product_key` esistono nel DB
- Se `billing_country` = `IT`: almeno uno tra `pec_email` e `sdi_code` obbligatorio

Risposte:

```
// 200 - Dati di fatturazione salvati { "status": "saved", "vat_number": "IT12345678901" } // 401 - JWT non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 404 - Cliente non trovato { "error_code": "CLIENT_NOT_FOUND", "message": "Nessun cliente trovato per vat number e product key", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 422 - Dati fatturazione non validi { "error_code": "INVALID BILLING DATA", "message": "Per clienti italiani è obbligatorio pec_email oppure sdi_code", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F8 — POST /api/vendor/license/revoke

Scopo: Revoca la licenza attiva o provvisoria di un cliente (es. per mancato pagamento). Imposta `status` = `revoked`, invalida l'`offline_token` e mette in coda email + messaggio in-app di notifica al cliente. **Idempotente:** se la licenza è già revocata, la chiamata è un no-op e restituisce 200.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body:

```
{
  "vat_number": "1112345678901", // P.IVA del cliente — obbligatorio "product_key": "PK-XYZ", //
  "prodotto da revocare" — obbligatorio "reason": "Mancato pagamento" // Motivo revoca — opzionale, usato
  nel template email
}
```

Tabelle DB coinvolte: `clients` (lettura), `licenses` (aggiornamento `status`), `messages` (inserimento notifica)

Controlli:

- JWT vendor valido
- `vat_number` + `product_key` esistono nel DB
- Se licenza già revocata → no-op, risposta 200

Risposte:

```
// 200 - Licenza revocata { "status": "revoked", "license_key": "lk_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h",
  "revoked_at": "2026-06-09T12:00:00Z" } // 401 - JWT non valido { "error_code": "INVALID_JWT", "message":
  "Token non valido o scaduto", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" } // 404
  "Licenza non trovata" { "error_code": "LICENSE_NOT_FOUND", "message": "Nessuna licenza attiva trovata per
  vat number e product key", "timestamp": "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

F9 — POST `/api/vendor/auth/rotate-key` (nuovo in v4)

Scopo: Genera una nuova API key per il vendor e revoca quella corrente. La vecchia chiave rimane valida per un grace period (es. 1h) per permettere la transizione senza downtime. Lo storico delle chiavi viene salvato in `vendors.api_key_history`.

Header:

```
Authorization: Bearer <jwt_vendor>
```

Request body: nessuno

Tabelle DB coinvolte: `vendors` (aggiornamento `api_key_hash`, `api_key_revoked_at`, `api_key_history`)

Controlli:

- JWT vendor valido

Risposte:

```
// 200 - Nuova API key generata { "new_api_key": "vk_nuova_chiave_in_chiaro_mostratasoloora", //
  Mostrata una sola volta "old key valid until": "2026-06-09T13:00:00Z" // Grace period: 1h } // 401 - JWT
  non valido { "error_code": "INVALID_JWT", "message": "Token non valido o scaduto", "timestamp":
  "2026-06-09T10:00:00Z", "request_id": "uuid" }
```

CHIAMATA USCENTE (O1)

01 — GET {vendor_erp_url}/alarm

Scopo: Chiamata uscente dal Service Invoice verso l'ERP del fornitore. Notifica eventi rilevanti (nuova registrazione, licenza in scadenza, licenza scaduta). **Viene eseguita esclusivamente dai job schedulati del sistema eventi (sezione 12) — mai in risposta diretta a una chiamata API.** L'URL di destinazione è configurato in `vendors.erp_alarm_url`.

Chiamata HTTP uscente (il Service Invoice chiama l'ERP):

```
GET {vendors.erp_alarm_url}/alarm?alarm_code=NEW_REGISTRATION
&license_key=1k_6f8d3c1a2b9e4f5c8a1b2c3d4e5f6g7h_&vat_number=IT12345678901_&company_name=Acme+Sr
&product_key=PK-XYZ_&timestamp=2026-06-09T12:00:00Z
```

Valori possibili di `alarm_code`:

Valore	Descrizione	Job che lo attiva
NEW_REGISTRATION	Nuovo cliente registrato (<code>vendor_synced = false</code>)	NEW_REGISTRATION
LICENSE_EXPIRING	Licenza in scadenza entro soglia configurata	LICENSE_EXPIRING
LICENSE_EXPIRED	Licenza scaduta oggi	LICENSE_EXPIRED

Comportamento in base alla risposta dell'ERP:

```
HTTP 200 OK → alarm_logs: success = true HTTP altro → alarm_logs: success = false, retry_count = 0 Job
ALARM_RETRY riprovera (max 3 tentativi) Al 3° fallimento → email GET_ALARM_FALLBACK al fornitore
alarm_logs: permanently_failed = true
```

Tabelle DB coinvolte: `alarm_logs` (inserimento/aggiornamento), `licenses` (aggiornamento `vendor_synced` se successo)

RIEPILOGO TABELLE DB — Campi principali

`vendors`

Campo	Tipo	Descrizione
<code>id</code>	INT PK	ID vendor
<code>name</code>	VARCHAR(255)	Nome fornitore

api_key_hash	VARCHAR(255)	Hash bcrypt dell'API key corrente
api_key_revoked_at	TIMESTAMP	Data revoca (NULL se attiva)
api_key_history	JSON	Storico chiavi precedenti
erp_alarm_url	VARCHAR(500)	URL endpoint O1 dell'ERP
created_at	TIMESTAMP	

vendor_general_setup (nuova — sezione 12)

Campo	Tipo	Descrizione
id	INT PK DEFAULT 1	Sempre 1 — record unico
vendor_id	INT FK	Vendor di questa istanza
default_check_interval_hours	INT DEFAULT 24	Frequenza default job schedulati

vendor_event_config (nuova — sezione 12)

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
event_code	VARCHAR(50)	Es. NEW_REGISTER

enabled	BOOLEAN DEFAULT true	ON = job attivo, OFF = job non registrato
check_interval_hours	INT NULL	NULL = usa default_che
settings_json	TEXT	Config specifica (soglie, max_retries.
last_run_at	TIMESTAMP	Ultimo avvio job

products

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
product_key	VARCHAR(50) UNIQUE	Chiave da distribuire nella libreria client
product_name	VARCHAR(255)	
trial_duration_days	INT	
trial_max_users	INT	

license_check_frequency_days	INT	Frequenza check C4
------------------------------	-----	--------------------------

clients

Campo	Tipo	Descrizione
id	INT PK	
vendor_id	INT FK	
product_id	INT FK	
vat_number	VARCHAR(50)	
company_name	VARCHAR(255)	
country	VARCHAR(10)	Codice ISO
contact_email	VARCHAR(255)	
contact_phone	VARCHAR(50)	
referent_name	VARCHAR(255)	
status	ENUM(pending, active)	
last_c5_at	TIMESTAMP	Ultima chiamata C5 (per inattività)
inactivity_notified_at	TIMESTAMP	Ultima notifica CLIENT_INA

licenses

Campo	Tipo	Descrizione
id	INT PK	

client_id	INT FK	
license_key	VARCHAR(255) UNIQUE	Generata con HMAC-SHA256 + salt
license_type	ENUM(trial, monthly, annual, provisional)	
status	ENUM(active, expired, revoked)	
starts_at	TIMESTAMP	
expires_at	TIMESTAMP	
max_users	INT	
vendor_synced	BOOLEAN DEFAULT false	true dopo conferma O1 (gestito dal job)
offline_token	TEXT	Crittografato con AES-256-GCM
offline_token_expires_at	TIMESTAMP	
alarm_logs		
Campo	Tipo	Descrizione
id	INT PK	
license_id	INT FK	
alarm_code	VARCHAR(50)	NEW_REGISTERED, LICENSE_EXPIRED, LICENSE_EXPIRED
success	BOOLEAN	
retry_count	INT DEFAULT 0	

last_retry_at	TIMESTAMP
next_retry_at	TIMESTAMP
max_retries	INT DEFAULT 3
permanently_failed	BOOLEAN DEFAULT false
created_at	TIMESTAMP

true
dopo
max_retries
esauriti

Documento v5 — 09/06/2026 — basato su analisi v4 + decisione Alvise (sezione 12)